Synthetic customer data

Builds Japanese customer records from the real reference data in `bronze/reference` (names, companies, towns; see `SOURCES.md`). One entity can appear under several CONT_IDs, each showing one of its names / phones / addresses, with realistic typos and correlated missing values.

- **Configuration:** `SPEC` (entities, noise, missingness, column schema) and `SPLIT`
- **Engine:** `generate(spec)` builds the population; `check` asserts its invariants
- **Generate:** the population is split by entity into train / val / test

Paths come from `config/settings.toml`. Writes `silver/<split>.csv` (19 columns) and `labels/<split>.csv` (truth).

In [1]:
```python
import json
import unicodedata
from collections.abc import Mapping, Sequence
from dataclasses import dataclass, replace
from typing import Literal

import numpy as np
import pandas as pd

from er.settings import load_settings

Kind = Literal["person", "company"]


@dataclass(frozen=True, kw_only=True)
class Column:
    name: str
    dtype: str = "string"          # pandas dtype; codes stay string to keep leading zeros
    nullable: bool = True          # nullable columns share the rate_start → rate_end ramp, in column order
    group: str | None = None       # columns in one group share missingness (Spec.nesting), in column order
    kind: Kind | None = None       # filled only on rows of this kind; None = every row
    digits: int | None = None      # exact digit count, checked after generation
```

```
@dataclass(frozen=True)
class AddressFormat:
    template: str                # fields: pref city town chome_jp ban go five building room
    fullwidth: bool = False      # convert ASCII digits and hyphens to full width


@dataclass(frozen=True, kw_only=True)
class Spec:
    seed: int
    entities: int
    company_share: float
    dup_lambda: float                          # extra CONT_IDs per entity, Poisson
    multi_lambda: Mapping[Kind, float]         # extra phones / addresses / names per entity, Poisson
    cif_keep: float                            # chance a duplicate record reuses the entity's CIF_ID
    rate_start: float
    rate_end: float
    phone_prefixes: tuple[str, ...]
    phone_digits: int
    ban_max: int
    go_max: int
    address_pool_ratio: float                  # pool size / addresses held, per prefecture; < 1 makes unrelated ent
ities share
    full_name_sep: str
    original_name_sep: str
    legal_form_kana: str
    legal_prefix_share: float                  # 株式会社〇〇 vs 〇〇株式会社
    name_typo_rate: float                      # per record and name part (surname, given name, company name)
    kana_confusions: tuple[tuple[str, str], ...]  # swapped both ways; a dropped character when none applies
    phone_typo_rate: float                     # per record: one wrong PHONE_NUM_4 digit
    address_typo_rate: float                   # per record: one wrong digit (else a dropped character) in ORIGINAL_
ADDRESS
    nesting: Mapping[str, float]               # per group: 1 = fully nested missingness, 0 = independent
    buildings: tuple[str, ...]
    original_address_formats: tuple[AddressFormat, ...]
    columns: tuple[Column, ...]

    def __post_init__(self) -> None:
        names = [c.name for c in self.columns]
        assert len(set(names)) == len(names), "duplicate column names"
        assert 0 < self.rate_end <= self.rate_start <= 1
```

```
        for g in self.groups:
            assert len({c.kind for c in g}) == 1, f"group mixes kinds: {[c.name for c in g]}"
        assert set(self.nesting) <= {c.group for c in self.columns}, "nesting names an unknown group"
        assert all(0 <= rho <= 1 for rho in self.nesting.values())

    @property
    def rates(self) -> dict[str, float]:
        names = [c.name for c in self.columns if c.nullable]
        return dict(zip(names, np.linspace(self.rate_start, self.rate_end, len(names)).tolist()))

    @property
    def groups(self) -> list[list[Column]]:  # ungrouped nullable columns stand alone
        out: dict[str, list[Column]] = {}
        for c in self.columns:
            if c.nullable:
                out.setdefault(c.group or c.name, []).append(c)
        return list(out.values())


@dataclass(frozen=True, kw_only=True)
class Split:
    shares: Mapping[str, float]               # by entity, so a duplicate group never spans two splits
    seed: int

    def __post_init__(self) -> None:
        assert abs(sum(self.shares.values()) - 1) < 1e-9, "split shares must sum to 1"
```

Configuration

In [2]:

```
S = load_settings()

SPEC = Spec(
    seed=0,
    entities=40_000,                # large enough that 0 FP on test bounds the FP rate tightly
    company_share=0.2,
    dup_lambda=0.25,
    multi_lambda={"person": 0.3, "company": 1.0},
    cif_keep=0.5,
    rate_start=0.9999,
```

```
    rate_end=0.6544,
    phone_prefixes=("90", "80"),   # 090 / 080 mobiles with the leading 0 dropped
    phone_digits=10,
    ban_max=30,
    go_max=30,
    address_pool_ratio=0.5,
    full_name_sep="",
    original_name_sep=" ",
    legal_form_kana="カブシキガイシャ",
    legal_prefix_share=0.5,
    name_typo_rate=0.1,
    kana_confusions=(("ウ", "ー"), ("オ", "ウ"), ("ツ", "ッ"), ("ヤ", "ャ"), ("ユ", "ュ"), ("ヨ", "ョ"), ("ガ", "カ"),
                     ("ギ", "キ"), ("グ", "ク"), ("ゴ", "コ"), ("ジ", "シ"), ("ダ", "タ"), ("ド", "ト"), ("ブ", "フ")),
    phone_typo_rate=0.05,
    address_typo_rate=0.05,
    nesting={"name": 0.7, "postal": 0.9, "address": 0.6},
    buildings=("サクラハイツ", "メゾン青葉", "グランドール", "コーポ山田", "パークタワー"),
    original_address_formats=(
        AddressFormat("{pref}{city}{town}{chome_jp}{ban}番{go}号"),
        AddressFormat("{city}{town}{five}", fullwidth=True),
        AddressFormat("{pref}{city}{town}{five} {building}{room}"),
    ),
    columns=(
        Column(name="CONT_ID", nullable=False, digits=10),
        Column(name="PHONE_NUM_4", digits=4),
        Column(name="CIF_ID", digits=13),
        Column(name="FULL_NAME_KANA", group="name", kind="person"),
        Column(name="O_PERSON_NAME_KANA", group="name", kind="person"),
        Column(name="POST_CD", group="postal", digits=7),
        Column(name="POST_CD_KANA", group="postal", digits=7),
        Column(name="LAST_NAME_KANA", group="name", kind="person"),
        Column(name="GIVEN_NAME_KANA", group="name", kind="person"),
        Column(name="NYS_CNV_P_NAME_KANA", kind="company"),
        Column(name="SOURCE_ADDR_CD", group="address", digits=10),
        Column(name="ORIGINAL_ADDRESS", group="address"),
        Column(name="ADDRESS_FIVE", group="address"),
        Column(name="ADDRESS_FIVE_KANA", group="address"),
        Column(name="ADDR_CD_KANA", group="address", digits=11),
        Column(name="ADDR_CD", group="address", digits=11),
        Column(name="STD_ADDRESS", group="address"),
        Column(name="ADDRESS_ONE", group="address"),
```

```
        Column(name="ADDRESS_TWO", group="address"),
    ),
)

SPLIT = Split(shares={"train": 0.6, "val": 0.2, "test": 0.2}, seed=0)
assert set(SPLIT.shares) == set(S.data.datasets), "split names must match config/settings.toml"
```

Engine

In [3]:

```
FULLWIDTH = str.maketrans("0123456789−", "０１２３４５６７８９－")


def load_reference(folder: Path) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    def read(name: str) -> pd.DataFrame:
        return pd.DataFrame(json.loads((folder / name).read_text(encoding="utf-8")))
    persons, companies, towns = read("individual.json"), read("companies.json"), read("addresses.json")
    towns["chome"] = pd.Series([None if pd.isna(c) else int(c) for c in towns["chome"]], dtype=object)  # int, or None (no 丁目)
    towns = towns.drop_duplicates(["city_code", "town", "chome"], ignore_index=True)  # geolonia repeats some towns per 小字
    towns["town_seq"] = towns.groupby("city_code")["town"].transform(lambda t: pd.factorize(t, sort=True)[0] + 1)
    assert towns["town_seq"].max() < 1000, "town sequence must fit 3 digits of ADDR_CD"
    return persons, companies, towns


@dataclass(frozen=True)
class Address:
    town: int                   # row in the town reference
    ban: int
    go: int
    code: str                   # SOURCE_ADDR_CD


@dataclass
class Generated:
    df: pd.DataFrame            # pipeline columns, with missing cells
    clean: pd.DataFrame         # pipeline columns before blanking
    with_truth: pd.DataFrame    # clean + ENTITY_ID, KIND
    truth: pd.DataFrame
```

```
    applies: dict[str, pd.Series]
    typos: dict[str, int]


def generate(spec: Spec, persons: pd.DataFrame, companies: pd.DataFrame, towns: pd.DataFrame) -> Generated:
    rng = np.random.default_rng(spec.seed)
    digits = {c.name: c.digits for c in spec.columns}
    confusions = [(a, b) for x, y in spec.kana_confusions for a, b in ((x, y), (y, x))]
    typos = {"name": 0, "phone": 0, "address": 0}
    town_rows = towns.to_dict("records")
    surname_pool = persons["surname_kana"].to_numpy()

    def pick[T](xs: Sequence[T]) -> T:
        return xs[rng.integers(len(xs))]

    def distinct_digits(n: int, k: int) -> list[str]:  # n distinct k-digit numbers
        return [str(x) for x in 10 ** (k - 1) + rng.choice(9 * 10 ** (k - 1), n, replace=False)]

    def phone() -> str:
        prefix = pick(spec.phone_prefixes)
        rest = spec.phone_digits - len(prefix)
        return f"{prefix}{rng.integers(10 ** rest):0{rest}d}"

    def kana_typo(s: str) -> str:  # one realistic kana error: a confusable swap, else a dropped character
        spots = [(i, b) for i, ch in enumerate(s) for a, b in confusions if ch == a]
        if spots:
            i, b = pick(spots)
            return s[:i] + b + s[i + 1:]
        return drop_one(s)

    def drop_one(s: str) -> str:  # drop a non-initial character; 1-character values (e.g. the surname 1) stay int
act
        if len(s) < 2:
            return s
        i = int(rng.integers(1, len(s)))
        return s[:i] + s[i + 1:]

    def digit_typo(s: str) -> str:  # one wrong digit, keeping its width; else a dropped character
        spots = [i for i, ch in enumerate(s) if ch.isdigit()]
        if not spots:
            return drop_one(s)
```

```
        i = pick(spots)
        d = (unicodedata.digit(s[i]) + int(rng.integers(1, 10))) % 10
        return s[:i] + (chr(0xFF10 + d) if ord(s[i]) >= 0xFF10 else str(d)) + s[i + 1:]

    def noisy(s: str, kind: Literal["name", "phone", "address"]) -> str:
        rate = {"name": spec.name_typo_rate, "phone": spec.phone_typo_rate, "address": spec.address_typo_rate}[kin
d]
        if rng.random() >= rate:
            return s
        typos[kind] += 1
        return kana_typo(s) if kind == "name" else digit_typo(s)

    def five(a: Address) -> str:  # 丁目-番-号, or 番-号 where the town has no 丁目
        chome = town_rows[a.town]["chome"]
        return f"{chome}-{a.ban}-{a.go}" if chome is not None else f"{a.ban}-{a.go}"

    def address_columns(a: Address) -> dict[str, str]:
        t = town_rows[a.town]
        addr_cd = f"{t['city_code']}{t['town_seq']:03d}{t['chome'] or 0:03d}"  # 住所コード: municipality + town no.
+ chome
        return dict(POST_CD=t["postal"], POST_CD_KANA=t["postal"], SOURCE_ADDR_CD=a.code, ADDRESS_FIVE=five(a),
                    ADDRESS_FIVE_KANA=five(a), ADDR_CD_KANA=addr_cd, ADDR_CD=addr_cd,
                    STD_ADDRESS=f"{t['pref']}{t['city']}{t['town']}{five(a)}", ADDRESS_ONE=t["pref"], ADDRESS_TWO=
t["city"])

    def original_address(a: Address) -> str:  # uncleaned, the way it was typed in
        t = town_rows[a.town]
        building, room = pick(spec.buildings), f"{rng.integers(1, 10)}0{rng.integers(1, 10)}"
        fmt = pick(spec.original_address_formats)
        s = fmt.template.format(pref=t["pref"], city=t["city"], town=t["town"], ban=a.ban, go=a.go, five=five(a),
                                chome_jp=f"{t['chome']}丁目" if t["chome"] is not None else "", building=building,
room=room)
        return noisy(s.translate(FULLWIDTH) if fmt.fullwidth else s, "address")

    kinds = np.where(rng.random(spec.entities) < spec.company_share, "company", "person")
    lam = np.array([spec.multi_lambda[k] for k in kinds])
    n_phone, n_addr, n_name = (1 + rng.poisson(lam) for _ in range(3))
    n_rec = 1 + rng.poisson(spec.dup_lambda, spec.entities)
    assert spec.entities <= len(persons), f"{spec.entities:,} entities need as many distinct personas; there are
{len(persons):,}"
    # without replacement: two entities drawn from one persona would be clones (same name and prefecture), which n
```

```
o
    # matcher can tell apart. Companies use it only for a population-weighted prefecture.
    persona = rng.choice(len(persons), size=spec.entities, replace=False)
    pref = persons["prefecture"].to_numpy()[persona]

    # Address pool per prefecture, smaller than what its entities hold, so unrelated entities sometimes share (hou
seholds)
    towns_in = {p: g.index.to_numpy() for p, g in towns.groupby("pref")}
    assert set(pref) <= set(towns_in), f"prefectures without towns: {set(pref) - set(towns_in)}"
    need = pd.Series(n_addr).groupby(pref).sum()
    codes = iter(distinct_digits(int(need.sum()), digits["SOURCE_ADDR_CD"]))
    pools: dict[str, list[Address]] = {}
    for p, k in need.items():
        t = towns_in[p]
        shape = (len(t), spec.ban_max, spec.go_max)
        size = min(max(1, int(k * spec.address_pool_ratio)), int(np.prod(shape)))
        ti, b, g = np.unravel_index(rng.choice(np.prod(shape), size, replace=False), shape)
        pools[p] = [Address(int(t[i]), int(bb) + 1, int(gg) + 1, next(codes)) for i, bb, gg in zip(ti, b, g)]

    cifs = iter(distinct_digits(spec.entities + n_rec.sum(), digits["CIF_ID"]))
    company_names = companies["name_kana"].to_numpy()
    rows: list[dict[str, str]] = []
    for e, kind in enumerate(kinds):
        phones = [phone() for _ in range(n_phone[e])]
        pool = pools[pref[e]]
        addrs = rng.choice(len(pool), min(n_addr[e], len(pool)), replace=False).tolist()
        if kind == "person":  # extra surnames (maiden / married) come from the persona surname pool
            names, given = [persons.at[persona[e], "surname_kana"]], persons.at[persona[e], "given_kana"]
            while len(names) < n_name[e]:
                s = pick(surname_pool)
                names += [s] if s not in names else []
        else:  # (name, legal form in front?)
            names = [(s, rng.random() < spec.legal_prefix_share) for s in rng.choice(company_names, n_name[e], rep
lace=False).tolist()]
        base_cif = next(cifs)
        for r in range(n_rec[e]):
            a = pool[pick(addrs)]
            row = dict(ENTITY_ID=f"E{e:06d}", KIND=str(kind), PHONE_NUM_4=noisy(pick(phones)[-digits["PHONE_NUM_
4"]:], "phone"),
                       CIF_ID=base_cif if r == 0 or rng.random() < spec.cif_keep else next(cifs),
                       ORIGINAL_ADDRESS=original_address(a), **address_columns(a))
```

```
            if kind == "person":  # typos go into the parts, so every name column agrees
                last, first = noisy(pick(names), "name"), noisy(given, "name")
                row |= dict(FULL_NAME_KANA=f"{last}{spec.full_name_sep}{first}",
                            O_PERSON_NAME_KANA=f"{last}{spec.original_name_sep}{first}",
                            LAST_NAME_KANA=last, GIVEN_NAME_KANA=first)
            else:
                stem, legal_first = pick(names)
                stem = noisy(stem, "name")
                row["NYS_CNV_P_NAME_KANA"] = spec.legal_form_kana + stem if legal_first else stem + spec.legal_for
m_kana
            rows.append(row)

    with_truth = pd.DataFrame(rows).sample(frac=1, random_state=rng, ignore_index=True)  # shuffle so duplicates a
ren't adjacent
    with_truth.insert(0, "CONT_ID", [f"{i:0{digits['CONT_ID']}d}" for i in range(1, len(with_truth) + 1)])
    clean = with_truth.reindex(columns=[c.name for c in spec.columns]).astype({c.name: c.dtype for c in spec.colum
ns})
    truth = with_truth[["CONT_ID", "ENTITY_ID", "KIND"]].astype("string")

    # A cell stays filled iff its score ranks below round(rate · n), so every rate is exact.
    # score = ρ · (per-row group draw) + (1 − ρ) · (per-cell draw): ρ = 1 nests the group fully.
    rates = spec.rates
    applies = {c.name: with_truth["KIND"].eq(c.kind) if c.kind else pd.Series(True, index=with_truth.index) for c
in spec.columns}
    df = clean.copy()
    for group in spec.groups:
        idx = df.index[applies[group[0].name]]
        rho = spec.nesting.get(group[0].group, 1.0)
        shared = rng.random(len(idx))
        for c in group:
            score = rho * shared + (1 - rho) * rng.random(len(idx))
            rank = pd.Series(score.argsort().argsort(), index=idx)
            df.loc[rank.index[rank >= round(rates[c.name] * len(idx))], c.name] = None
    return Generated(df, clean, with_truth, truth, applies, typos)


def check(spec: Spec, g: Generated) -> None:
    rates = spec.rates
    for c in spec.columns:
        applies = g.applies[c.name]
        assert g.df[c.name].dtype == c.dtype, c.name
```

```
        assert g.clean[c.name].notna().equals(applies), f"{c.name}: clean data must be filled exactly where it app
lies"
        if c.digits:
            assert g.df[c.name].dropna().str.fullmatch(rf"\d{{{c.digits}}}").all(), c.name
        expected = round(rates[c.name] * applies.sum()) if c.nullable else applies.sum()
        assert g.df[c.name].notna().sum() == expected, c.name
    for group in spec.groups:
        if spec.nesting.get(group[0].group, 1.0) == 1:
            for prev, c in zip(group, group[1:]):
                assert not (g.df[c.name].notna() & g.df[prev.name].isna()).any(), (prev.name, c.name)
    assert g.df["CONT_ID"].is_unique
    assert g.with_truth.groupby("CIF_ID")["ENTITY_ID"].nunique().max() == 1  # no CIF_ID shared by two entities
    assert g.with_truth.groupby("SOURCE_ADDR_CD")["STD_ADDRESS"].nunique().max() == 1
    assert g.with_truth.groupby("STD_ADDRESS")["SOURCE_ADDR_CD"].nunique().max() == 1
```

Generate the population

The asserts in `check` run on the whole population, before it is split.

In [4]:
```
persons, companies, towns = load_reference(S.paths["reference"])
print(f"reference: {len(persons):,} personas · {len(companies):,} companies · {len(towns):,} towns in {towns['pre
f'].nunique()} prefectures")
main = generate(SPEC, persons, companies, towns)
check(SPEC, main)
ent = main.truth.groupby("ENTITY_ID").size()
print(f"{len(main.df):,} records · {len(ent):,} entities · {(ent > 1).mean():.1%} duplicated · typos {main.typo
s}")
print(pd.DataFrame({"target": SPEC.rates, "actual": {c: main.df.loc[main.applies[c], c].notna().mean() for c in SP
EC.rates}}).round(4).T)
main.df.head()
```

```
reference: 60,000 personas · 31,596 companies · 41,567 towns in 47 prefectures
```

```
50,215 records · 40,000 entities · 22.5% duplicated · typos {'name': 9099, 'phone': 2535, 'address': 2528}
        PHONE_NUM_4  CIF_ID  FULL_NAME_KANA  O_PERSON_NAME_KANA  POST_CD  \
target       0.9999  0.9796          0.9593              0.9389   0.9186
actual       0.9999  0.9796          0.9593              0.9389   0.9186

        POST_CD_KANA  LAST_NAME_KANA  GIVEN_NAME_KANA  NYS_CNV_P_NAME_KANA  \
target        0.8983           0.878           0.8576               0.8373
actual        0.8983           0.878           0.8576               0.8373

        SOURCE_ADDR_CD  ORIGINAL_ADDRESS  ADDRESS_FIVE  ADDRESS_FIVE_KANA  \
target           0.817            0.7967        0.7763              0.756
actual           0.817            0.7967        0.7763              0.756

        ADDR_CD_KANA  ADDR_CD  STD_ADDRESS  ADDRESS_ONE  ADDRESS_TWO
target        0.7357   0.7154       0.6950       0.6747       0.6544
actual        0.7357   0.7154       0.6951       0.6747       0.6544
```

Out[4]:

	CONT_ID	PHONE_NUM_4	CIF_ID	FULL_NAME_KANA	O_PERSON_NAME_KANA	POST_CD	POST_CD_KANA	LAST_NAME_KANA	GIVEN_NAME_KANA	NYS_CNV_P_NAME_KANA	SOURCE_ADDR_CD	ORIGINAL_ADDRESS	ADDRESS_FIVE	ADDRESS_FIVE_KANA	ADDR_CD_KANA	ADDR_CD	STD_ADDRESS	ADDRESS_ONE	ADDRESS_TWO
0	0000000001	3483	9781120565631	ナカニシコユキ	ナカニシ ココユキ	4228036	4228036	ナカニシ	コユキ	<NA>	<NA>	静岡県静岡市駿河区敷地1丁目23番6号	<NA>	1-23-6	<NA>	<NA>	<NA>	<NA>	<NA>
1	0000000002	2759	4287580443897	<NA>	<NA>	3650033	3650033	<NA>	<NA>	カブシキガイシャアイアイメディガル	3392893435	鴻巣市生出塚２－２３－１０	2-23-10	<NA>	11217020002	<NA>	<NA>	<NA>	鴻巣市
2	0000000003	7855	5096367648455	ババケイト	ババ ケイト	8113304	8113304	ババ	ケイト	<NA>	1677043453	福岡県福津市津屋崎7-2-1 グランドール206	7-2-1	7-2-1	40224011007	40224011007	福岡県福津市津屋崎7-2-1	福岡県	福津市
3	0000000004	6288	4648826056971	ヨシノショウシ	ヨシノ ショウシ	<NA>	<NA>	ヨシノ	ショウシ	<NA>	6133307438	<NA>	12-7	<NA>	01223005000	<NA>	<NA>	<NA>	<NA>
4	0000000005	3316	3857655848458	ヤマノウエタカシ	ヤマノウエタカシ	1130033	1130033	ヤマノウエ	<NA>	<NA>	7659677116	文京区本郷１－２３－２１	1-23-21	1-23-21	13105009001	13105009001	東京都文京区本郷1-23-21	東京都	文京区

Split into train / val / test by entity

Exact shares; every duplicate group stays inside one split.

In [5]:
```
entities = np.random.default_rng(SPLIT.seed).permutation(main.truth["ENTITY_ID"].unique())
cuts = np.round(np.cumsum(list(SPLIT.shares.values())) * len(entities)).astype(int)[:-1]
split_of = {e: name for name, part in zip(SPLIT.shares, np.split(entities, cuts)) for e in part}
split = main.truth["ENTITY_ID"].map(split_of).rename("split")

assert split.notna().all() and main.truth.assign(split=split).groupby("ENTITY_ID")["split"].nunique().max() == 1
S.data.silver_dir.mkdir(parents=True, exist_ok=True)
S.data.labels_dir.mkdir(parents=True, exist_ok=True)
for name, share in SPLIT.shares.items():
    rows_in = split == name
    assert abs(main.truth.loc[rows_in, "ENTITY_ID"].nunique() / len(entities) - share) <= 0.01, name
    main.df[rows_in].to_csv(S.data.silver_dir / f"{name}.csv", index=False)
    main.truth[rows_in].to_csv(S.data.labels_dir / f"{name}.csv", index=False)
print(split.value_counts().rename("records").to_frame().assign(entities=main.truth.groupby(split)["ENTITY_ID"].nun
ique()))
```

```
       records  entities
split
train    30147     24000
test     10066      8000
val      10002      8000
```